Visual reconstruction of played notes on a piano keyboard from a single top-view video

Flavio De Lellis Filippo Garofalo
Emanuele Turbanti

Image Analysis and Computer Vision – Politecnico di Milano

June 8, 2026

Abstract

We present a pipeline that takes an ordinary RGB video of two hands playing a piano, filmed roughly from above, and marks on every frame which keys are being pressed. The system uses only classical image processing and projective geometry, with no learning involved and a single short manual calibration. Our contribution over similar course projects is twofold. First, the keyboard is rectified through a homography that we estimate from scratch with a normalized Direct Linear Transform, and the key grid is then built analytically in millimetres so that both white *and* black keys are modelled. Second, the press decision combines a spatial cue (a finger lies on the key) with a temporal cue (the key surface actually changed), where the temporal stage uses a signed frame difference that treats white and black keys with opposite polarity. A key is reported as pressed only when both cues agree. We tested the method on five videos with very different lighting, backgrounds and hand styles.

1 Introduction

The goal of this project is simple to state and surprisingly hard to do well: given a video of someone playing the piano, recover which notes are sounding at each instant, using nothing but the picture. We deliberately stay with a single, fixed, roughly top-down camera, because that is the cheapest setup a student or a hobbyist already owns. The output is the original video with the pressed keys painted on top, keeping the link between each pressed key and the finger responsible for it visible.

The motivation comes mostly from Music Information Retrieval, where automatic transcription tries to turn a performance into a symbolic score or a MIDI file [1]. Audio based transcription is the usual route, but it has well known trouble with dense polyphony, with notes that overlap and with instruments that are slightly out of tune. Looking at the player's hands sidesteps part of that, because the physical contact between a finger and a key is unambiguous in a way that mixed audio is not. The same idea has obvious uses in tutoring: a system that can see a wrong finger or a wrong timing could give feedback to a student practising at home, which matters for remote teaching.

There is also a plain computer vision reason to care. Hands on a piano are about the worst case for hand tracking. They are not posed to help the camera, they occlude each other constantly, fingers curl down toward the keys so the fingertips disappear, and a finger can rest on a key without playing it. A method that copes with this is a method that copes with a lot.

2 Method

2.1 Overview

We stay deliberately on the cheap end of the hardware spectrum: a single RGB camera, no depth sensor and no second viewpoint, and no learned component anywhere in the chain. Adding hardware would make the problem easier (a frontal camera or a depth sensor measures key travel directly [2], and a CNN could absorb much of the appearance variation), but each of those options carries a cost, in equipment or in annotated data, that a setup meant to run on footage anyone can shoot should not assume. Working inside that constraint is the whole point, and Figure 1 shows how we do it.

The pipeline has a one-time calibration and then a per-frame loop. Calibration recovers the geometry of the keyboard and freezes a virtual key grid. The loop runs two parallel branches on every frame: a *spatial* branch that asks which keys have a fingertip over them, and a *temporal* branch that asks which keys have visually changed with respect to a resting reference. A key is declared pressed only when it appears in both branches at once. Figure 1 sketches the flow.

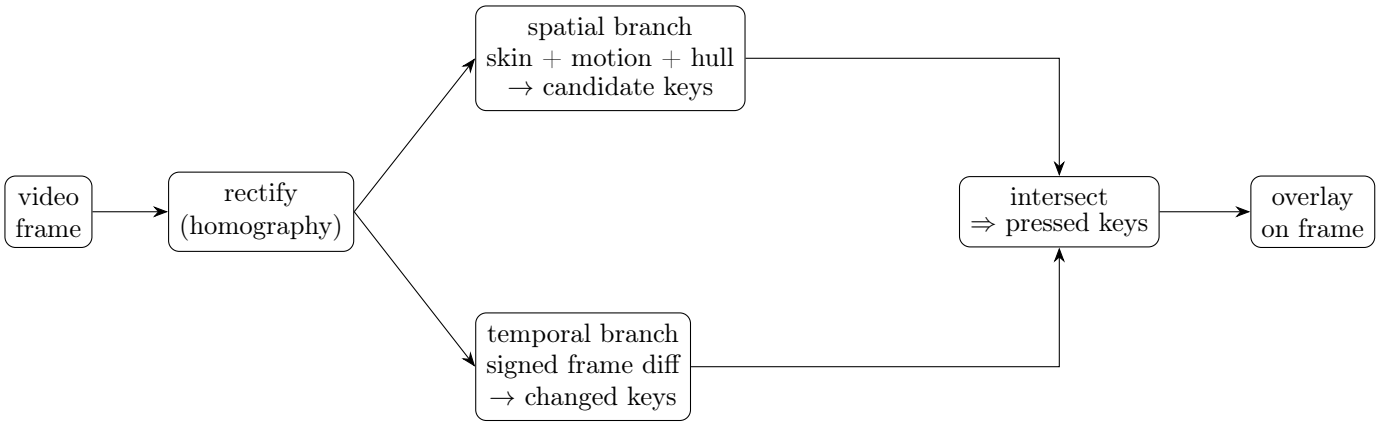


Figure 1: High level structure. Calibration (not shown) runs once and produces the homography and the virtual key grid that both branches rely on.

2.2 Geometric calibration and rectification

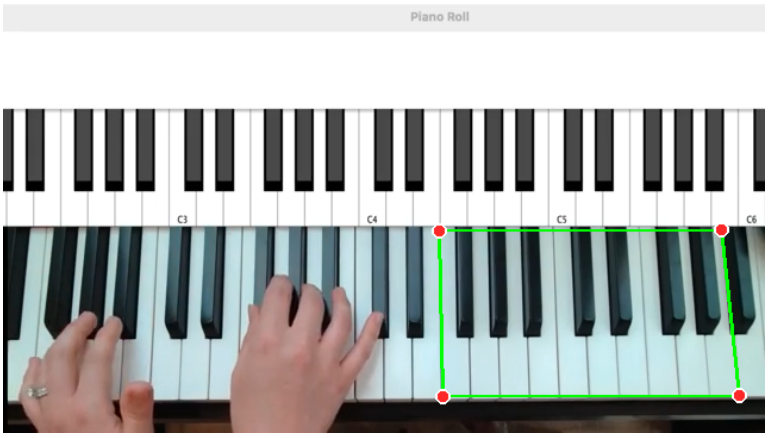
Because the camera sits at a slight angle to the keyboard normal, the keys appear as a trapezoid rather than a rectangle, and reasoning about key positions directly in pixels is fragile. We remove this distortion with a planar homography. The top faces of the white keys lie on a single plane π, and the image of that plane is related to the plane itself by

$$\mathbf{u} \sim \mathtt{H}\,\mathbf{x}, \qquad \mathbf{u} = [x, y, 1]^\top, \quad \mathbf{x} = [X, Y, 1]^\top, \tag{1}$$

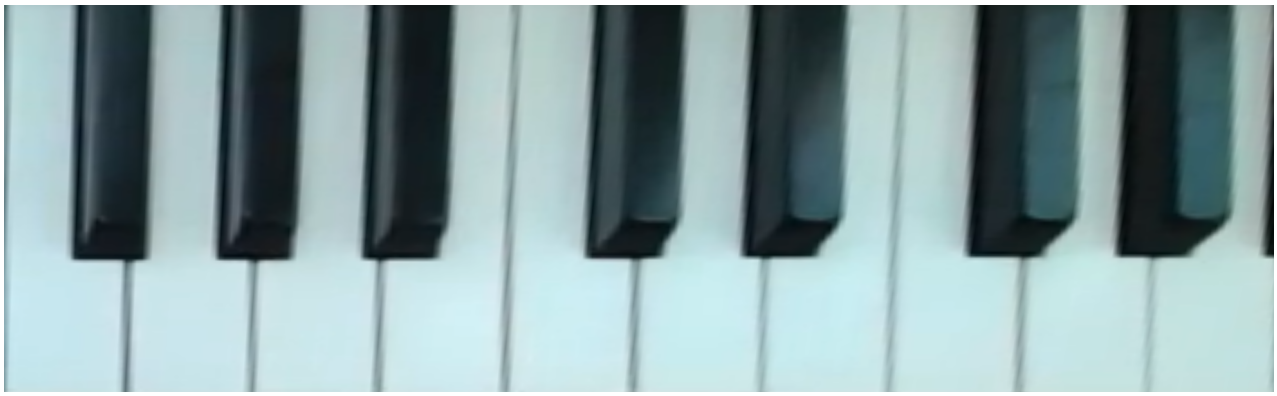
where $\mathbf{x}$ is a point on the keyboard plane in millimetres and $\mathbf{u}$ is its pixel in the frame. We do not call a library routine for this. The operator clicks the four corners of the visible white-key region once, those corners are ordered as top-left, top-right, bottom-right, bottom-left from the signs of their coordinate sums and differences, and they are matched to a metric rectangle whose width is (number of white keys) $\times 23.5$ mm and whose height is 150 mm.

The homography is then estimated with a Direct Linear Transform [3]. Each correspondence contributes two rows to a $2n \times 9$ matrix $\mathtt{A}$,

$$\begin{aligned} &[-X,\ -Y,\ -1,\ 0,\ 0,\ 0,\ xX,\ xY,\ x], \\ &[\,0,\ 0,\ 0,\ -X,\ -Y,\ -1,\ yX,\ yY,\ y], \end{aligned} \tag{2}$$



(a) Source frame with the four clicked corners (red) and the resulting white-key quadrilateral (green), which also serves as the region of interest.



(b) The same region after inverse warping with `H`: a metric, top-down view of the keys at 4 pixels per millimetre.

Figure 2: Calibration and rectification on a frame of one test video (21 white keys, leftmost note F). The homography straightens the slanted keyboard into the rectangular plane on which every later stage operates.

and the solution is the right singular vector of `A` associated with the smallest singular value, reshaped into the 3×3 matrix `H` and scaled so that $H_{33} = 1$. We keep both `H` and its inverse, since the two branches need opposite directions: rectifying an image needs plane to image, while mapping a detected fingertip back onto the keyboard needs image to plane. As a sanity check the calibration also reports the mean and maximum reprojection error of the four corners in pixels; on our videos this stayed well under one pixel, as expected when the input points are the same four points used to fit.

Rectification itself is done by inverse warping. We build the output grid in the metric plane at a chosen resolution (we used 4 pixels per millimetre), send each output pixel through `H` into the source frame, and sample there with bilinear interpolation. The result is a clean top-down image of the keys on which every later step operates. The four corners double as a region of interest mask: anything outside the clicked quadrilateral, in particular the red felt strip above the keys that otherwise pollutes skin detection, is discarded before segmentation. Figure 2 shows the result on one of our frames.

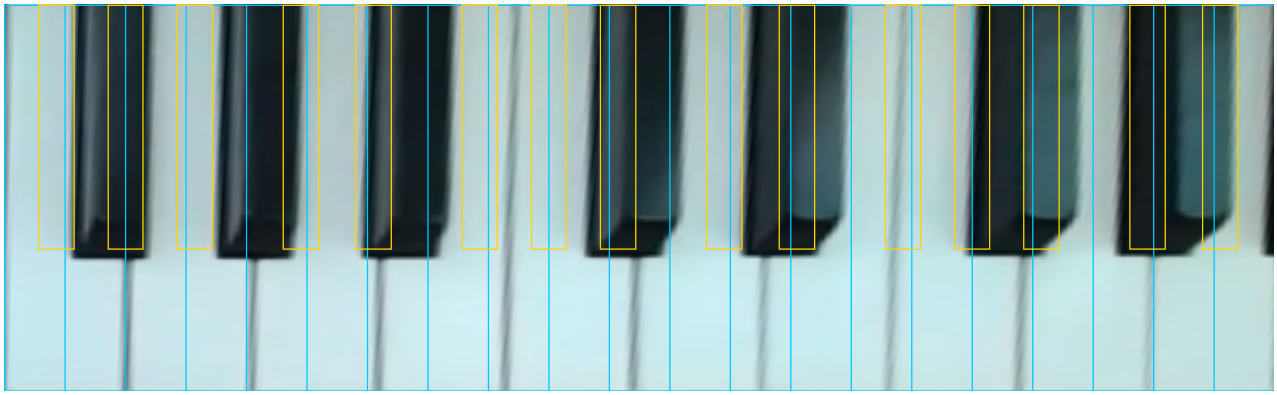


Figure 3: The virtual keyboard model drawn over the rectified plane: white-key columns in cyan and the 15 analytically placed black keys in yellow. The grid is computed from the two physical dimensions and the octave pattern, not measured from the image.

2.3 The virtual keyboard model

Once the plane is metric we never measure key positions from the image again. We generate them. White keys are an even partition of the calibrated width into equal columns. Black keys are placed analytically from the standard octave pattern [1 1 0 1 1 1 0] (reading the white notes C,D,E,F,G,A,B, a 1 meaning a black key sits to the right of that white). The pattern is rotated to start on the leftmost visible note, which the user supplies, and tiled across the visible range.

Inside each contiguous group of black keys (a pair for C#/D#, a triple for F#/G#/A#) we do not assume the blacks are centred on the white-white seams, because they are not. We require instead that every white key shows the same visible width below the black band, which gives, for a group of N blacks of width w_b on white keys of width w_w,

$$\text{gap} = w_w - \frac{N\, w_b}{N+1}, \qquad c_i = x_{\text{left}} + i \cdot \text{gap} + \left(i - \tfrac{1}{2}\right) w_b, \quad i = 1, \ldots, N, \tag{3}$$

where c_i is the centre of the i-th black key in the group. The constants ($w_w = 23.5$ mm, $w_b = 13.7$ mm, white length 150 mm, black length 95 mm) live in a single file, so changing the geometry propagates everywhere with no manual retuning. The end product is one array of keys, each carrying its polygon both in millimetres and in rectified pixels, its bounding box and its type. This is the shared map that the spatial and temporal branches both query. Handling black keys at all is something the reference projects we looked at left as future work, and the analytic placement is what made it tractable for us. Figure 3 overlays the generated grid on the rectified plane.

2.4 Spatial branch: from pixels to candidate keys

The spatial branch answers a narrow question, which keys have a finger over them, and it is allowed to over-answer, since the temporal branch will prune it.

Skin and motion. Skin is segmented in YCbCr, which separates luminance from chrominance and is a common choice for this exact task [4]. We keep pixels with $Y \in [100, 235]$, $C_b \in [85, 135]$, $C_r \in [135, 180]$. On its own this also fires on ivory keys, whose colour is close to pale skin, so we add a simple but effective constraint: hands move and keys do not. A motion mask is built by thresholding the absolute luminance difference between the current frame and the first frame (> 40 on the 0 to 255 scale), then closed and dilated to fill the hand. Intersecting skin with motion removes the static

ivory blobs that would otherwise turn into phantom fingers. The mask is then closed with a disk, hole-filled, and small components below 50 pixels are dropped.

Separating the two hands. We keep at most the two largest connected components, with an area floor of max(1000, $0.4 \times$ largest area). The relative term is what lets a real second hand through while still rejecting a distant reflection or shadow: a true second hand has area comparable to the first, a reflection does not.

Fingertips by convex hull. For each hand we compute the convex hull with a hand-written Graham scan [5], sorting points by polar angle around the lowest anchor and keeping only left turns. Fingertips are the hull vertices in the top half of the hand (above the median row), which are exactly the pointed extremities of the silhouette. Vertices closer than 20 pixels are merged so that one tip yields one point, and any vertex within 20 pixels of the frame border is discarded, because a hand cut off by the image edge produces hull corners that are artefacts, not fingertips. We are aware of the classic failure mode here, that curled fingers flatten the silhouette and the hull loses its peaks, and we discuss how the temporal stage absorbs the resulting misses in Section 3.

Mapping fingertips to keys. Each fingertip, detected in the original frame, is sent through the image-to-plane homography and scaled into rectified pixels, so it lands in the same coordinate system as the virtual keyboard. We then test black keys before white ones, because a black key physically sits on top of the rear portion of the whites, so a finger there is on the black key, not the white beneath it. The test is not exact, though, and the reason is parallax. The homography is calibrated on the plane of the white keys, but the black keys (and the knuckles that the hull tends to pick up) are a centimetre or so above that plane, so their projection lands further forward than it should. We compensate with vertical tolerances that we tuned on the videos: 80 pixels of slack when testing black-key boxes and 60 pixels when testing white-key polygons. A fingertip matches the first black box that contains it (within tolerance); otherwise it is shifted back and tested against the white polygons.

Candidate expansion across a chord. A finger rarely sits dead centre on a key, and a hand spanning a chord often triggers only some of the keys it actually covers. When two or more candidates are present we sort them by horizontal centre and split them into groups wherever the gap between consecutive candidates exceeds three white-key widths, which is how we detect the empty space between the left and the right hand. Each group is then filled in to span its full horizontal extent, so a hand resting across several keys reports all of them, while the gap between the two hands is left untouched.

2.5 Temporal branch: did the surface actually change?

The spatial branch cannot tell a pressed key from a key merely touched, because from above they look almost identical. The temporal branch resolves this by comparing each frame against the resting reference, which we take to be the first rectified frame. Crucially, the first frame is allowed to contain hands, as long as the keys are at rest, which makes the method usable on real footage that never starts on an empty keyboard.

We work with a *signed* grayscale difference, not the absolute one, and this is the part we are most happy with. Let $g = \text{gray(current)} - \text{gray(reference)}$. A finger is darker than a white key, so pressing a white key makes its surface get darker, $g < 0$. A finger or its skin is lighter than a black key, so a

black press makes the surface get lighter, $g > 0$. We therefore split the signal by sign:

$$d_{\text{white}} = \frac{\max(0, -g)}{255}, \qquad d_{\text{black}} = \frac{\max(0,\ g)}{255}. \tag{4}$$

The benefit is that the two key types stop interfering. When a white key is pressed, the darkening it produces, and the darkening that bleeds onto its neighbours, only ever feeds the white channel, so it cannot raise a false alarm on the black channel, and the other way round. The skin mask is subtracted from the white channel only, to stop the moving hand from registering as a press on nearby whites; on the black channel the skin *is* the useful signal, so we leave it.

Each channel is thresholded (20/255 for white, 25/255 for black), opened with a small disk to kill speckle, and then, for every key, we look at the fraction of "on" pixels inside its region. We shrink each region by a two pixel inner margin to reduce bleed from adjacent keys, and we declare the key changed when that fraction passes a per-type level (10% for white, 15% for black). Two extra refinements proved necessary in practice. First, the vertical band that we analyse is adaptive: instead of a fixed top third we cut just above the highest detected fingertip, with a floor of 15% of the keyboard height, which keeps the moving arm out of the analysis. Second, we shift the temporal regions slightly to the left (about 35 pixels) so that each key captures the shadow that a press casts to its side rather than the neighbour's.

We also implemented an earlier variant of this stage based on a weighted mean of the absolute difference, with a large weight on the white-to-black horizontal boundary and a Canny-based weight on the lateral edges. It works, but the signed two-channel version above is what the integrated system runs, because it generalises to black keys and needs less tuning.

2.6 Fusion

The two branches each return a set of key indices. The reported pressed keys are their intersection: a key must have a fingertip over it *and* have changed with respect to rest. This single AND is what removes the bulk of the false positives, since a hovering hand satisfies the spatial test but not the temporal one, and a shadow or reflection satisfies neither cleanly. The final overlay draws the spatial candidates as dashed outlines and the confirmed presses as filled regions, white presses in green and black presses in red, on top of the rectified frame.

3 Experiments and results

We recorded and collected five videos to stress different parts of the pipeline: four from public online sources and one filmed by us in a normal room. They vary in lighting, in background clutter, in whether the instrument is an acoustic grand or an electronic keyboard, and in hand morphology and posture. All of them start on a resting frame, which the temporal stage needs as its reference. Three are representative enough to describe here, and Figure 4 shows the final output on one of them.

The first video shows a baby grand under good, high-contrast light, but the camera is clearly off the keyboard normal, so the projective distortion is large. This makes it a good test of the calibration: once rectified, the keys line up with the virtual grid and the rest of the pipeline behaves as if the camera had been overhead. The pianist plays with a pronounced claw posture, so several fingertips are hidden and the convex hull misses them; the presses are still recovered because the temporal change on those keys is unambiguous, and the intersection keeps them as long as *some* fingertip in the group survived the expansion step.

The second video is an electronic keyboard against a busy, multicoloured background, with shadows between the black keys. The camera here is close to an ideal top-down view. This case mostly

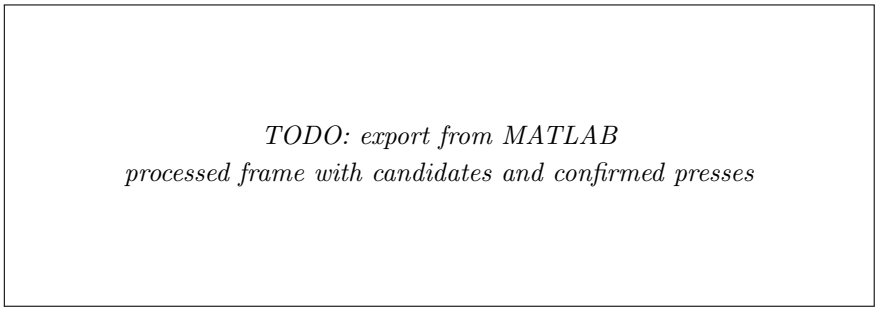


Figure 4: A processed frame. Dashed outlines are spatial candidates, filled regions are confirmed presses (green for white keys, red for black). (Export from MATLAB and replace.)

tests robustness of skin segmentation to background clutter; the motion gate is what carries it, since the colourful background is static and never enters the skin-and-motion intersection.

The third video is the one we filmed at home, and it is the hardest. Both hands play at once, the light is poor, and the wood of the instrument is close in colour to the skin, so chromatic contrast is low. This is the only case where we had to nudge the skin thresholds by hand. With that adjustment the system still separates the two hands correctly through the connected-component step and reports plausible presses on both.

Across the set, the pipeline marks pressed white keys reliably under projective distortion, cluttered backgrounds, low light and two-hand play, and it also picks up black-key presses, which the white-only reference systems do not attempt. The honest weak point is the candidate stage on its own: it flickers frame to frame, and in the hardest frames it produces spurious candidates. The temporal verification is what tames this, and the gap between the raw candidate stream and the final, intersected output is the clearest evidence that the two-cue design was the right call.

4 Conclusions and open problems

We built a single-camera, no-learning pipeline that reports pressed piano keys from a top-view video, using a from-scratch homography for rectification, an analytic virtual keyboard that includes black keys, a convex-hull spatial branch, and a signed two-channel temporal branch, fused by intersection. On a varied set of videos it handles white and black presses under a range of real conditions.

The limitations are the ones inherent to hand-tuned classical methods. Several thresholds (skin ranges, the motion gate, the parallax tolerances) depend on lighting and on the instrument, and a genuinely new scene can need a touch of retuning. A learned skin or hand model would remove much of that sensitivity, at the price of a labelled dataset we did not have. The detection is also purely frame-by-frame: adding tracking over time would let us reconstruct note onsets and durations, and from there an actual transcription. Finally the field of view covers only part of the keyboard at once, which caps the pitch range, and the black-key cue, while working, is more delicate than the white one because of the parallax we described in Section 2.5. These are the directions we would take the work next.

Contributions. Flavio handled the geometric calibration, the homography and the rectification. Filippo built the spatial branch, the skin and motion segmentation and the convex hull. Emanuele developed the temporal branch and the press confirmation. The fusion logic and the experiments were done jointly.

References

[1] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2019.

[2] M. Akbari and H. Cheng, "Real-time piano music transcription based on computer vision," in *IEEE Transactions on Multimedia*, vol. 17, pp. 2113–2121, 2015.

[3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 ed., 2004.

[4] S. L. Phung, A. Bouzerdoum, and D. Chai, "Color-based skin segmentation: an evaluation of the state of the art," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 1, pp. 148–154, 2005.

[5] R. L. Graham, "An efficient algorithm for determining the convex hull of a finite planar set," *Information Processing Letters*, vol. 1, no. 4, pp. 132–133, 1972.